

1 Problem Overview

The input consists of n integer values which are placed in a rectangular matrix with k columns. The placement follows a diagonal pattern controlled by the parameter r : the first r values occupy the last r cells of the first row, the next values continue in the second row starting from the leftmost column, and the pattern repeats every k rows. All other cells of the matrix contain 0.

We are given three integers p , q and r . The task is to obtain the largest possible sum of a region that is the union of

- a vertical strip of height q that spans the whole width k (i.e. rows $i - q + 1 \dots i$, all columns), and
- a horizontal strip of height 1 and width p that always lies in the **last** row of the matrix.

The two strips may intersect – when the bottom of the vertical strip coincides with the last row the intersection is a $p \times q$ rectangle. The goal is to maximise the total sum of the union.

2 Key Observations

- Because of the diagonal placement, each row of the matrix contains at most one non-zero entry. This makes the matrix extremely sparse, yet we must answer many rectangular sum queries.
- The sum of any axis-aligned rectangle can be obtained in $O(1)$ time if a two-dimensional prefix-sum table is available.
- The vertical strip V of height q has sum

$$\text{sum}(V) = P(i, k - 1) - P(i - q, k - 1),$$

where i is its bottom row and P is the prefix-sum array.

- The horizontal strip H of width p placed in the last row has sum

$$\text{sum}(H) = P(\text{lastLine}, j) - P(\text{lastLine}, j - p),$$

where j is the rightmost column of the strip.

- The sum of the union $V \cup H$ is given by the inclusion–exclusion formula

$$\text{sum}(V \cup H) = \text{sum}(V) + \text{sum}(H) - \text{sum}(V \cap H).$$

The intersection $V \cap H$ is a $p \times q$ rectangle only when $i = \text{lastLine}$; otherwise it is empty.

- When the p -wide strip is positioned on the left side of the matrix we can compute its sum by a complement trick:

$$\text{sum}(V \cup H) = \text{totalSum} - (P(\text{lastLine}, j) - P(\text{lastLine}, j - \text{colGap})) + (P(i, j) - P(i - q, j) - P(i, j - \text{colGap}) +$$

where $\text{colGap} = k - p$.

3 Solution

1. **Alignment.** Let `offset` = $k - r$. Shift all input values downwards by `offset` rows. After this operation the matrix has $\frac{n + \text{offset}}{k}$ rows (rounded up) and exactly k columns, and the diagonal described above starts in column $k - r$ of the first row.

2. **Prefix sums.** Using the same buffer a we build a 2-D prefix sum in-place:

$$a[i][j] += a[i-1][j] + a[i][j-1] - a[i-1][j-1] \quad (\text{for } i, j \geq 0).$$

After the construction $P(i, j) = \sum_{x=0}^i \sum_{y=0}^j a[x][y]$, so any rectangle sum can be obtained with the standard inclusion-exclusion formula.

3. **Total sum.** Keep `totalSum`, the sum of all input values; it is used in the complement computation.
4. **Enumeration of the vertical position.** Loop over all possible bottom rows i of the vertical strip. It must satisfy $i \geq q$ and the cell $(i, k-1)$ must exist. Its sum is

$$\text{lineSum} = P(i, k-1) - P(i-q, k-1).$$

5. **Right-aligned horizontal strip.** For each i iterate j from $p-1$ up to $k-1$ (the rightmost column of the strip). Compute

$$\begin{aligned} \text{colSum} &= P(\text{lastLine}, j) - P(\text{lastLine}, j-p), \\ \text{intersect} &= P(i, j) - P(i-q, j) - P(i, j-p) + P(i-q, j-p), \\ \text{cur} &= \text{lineSum} + \text{colSum} - \text{intersect}. \end{aligned}$$

Update the global maximum with `cur`.

6. **Left-aligned horizontal strip.** Let `colGap` = $k - p$. For the same i iterate j from `colGap` up to $k-2$. Using the complement formula compute

$$\text{hSum} = \text{totalSum} - (P(\text{lastLine}, j) - P(\text{lastLine}, j - \text{colGap})) + (P(i, j) - P(i-q, j) - P(i, j - \text{colGap}) + P(i-q, j - \text{colGap})).$$

Again update the maximum.

7. The loops cover every admissible pair (i, j) , therefore the maximum obtained is the answer.

4 Correctness

We prove that the algorithm always returns the maximum possible sum.

- **Coverage of all placements.** The vertical strip must have height q and span all k columns, hence its bottom row can be any i with $i \geq q$ and $i \leq \text{lastLine}$. The algorithm iterates over exactly this set of rows. The horizontal strip must have width p and lie in the last row. Its rightmost column j can be any integer with $p - 1 \leq j \leq k - 1$. The first inner loop enumerates precisely these j . For the left-aligned case the strip occupies the p columns that are **not** in the interval $[j - \text{colGap} + 1, j]$; the algorithm iterates over all possible j for which this interval is non-empty. Hence every feasible L-shaped region is examined.
- **Correctness of the sum formulas.** The prefix-sum array P satisfies $P(x, y) = \sum_{u \leq x, v \leq y} a[u][v]$. Therefore the difference $P(b, d) - P(a - 1, d) - P(b, c - 1) + P(a - 1, c - 1)$ equals the sum of the rectangle $[a..b] \times [c..d]$. Using this identity with appropriate corners yields the three formulas for `lineSum`, `colSum` and `intersect`. The inclusion–exclusion principle gives the exact sum of the union $V \cup H$.
- **Complement formula.** When the p -wide strip is placed on the left, the $k - p$ columns $[j - \text{colGap} + 1..j]$ are absent from the strip. Removing their contribution from the total sum and adding back the part that belongs to the vertical strip produces the same value as computing the union directly. Hence `hSum` equals `sum(V ∪ H)` for the left-aligned case.
- **Maximisation.** For each feasible placement the algorithm computes the exact sum of the corresponding region and keeps the largest of these values. Consequently the final answer is the maximum achievable sum.

5 Complexity Analysis

Let $N = n + \text{offset}$ be the total number of cells in the matrix (at most $1.1 \cdot 10^6$ in the constraints).

- Building the 2-D prefix sum touches each cell once: $O(N)$ time and $O(N)$ memory.
- The two nested loops iterate $(\text{lastLine} - q + 1) \times k = O(N)$ times, each iteration performing only a constant amount of work.

Thus the overall running time is $O(N)$ and the extra memory consumption is $O(N)$.

6 Reference Implementation

```
#include <fstream>
#include <iostream>
#include <math.h>
```

```

#include <limits.h>

using namespace std;

const int NMAX = 1e6 + 1;
const int OFFSET_MAX = 1e5 + 1;
int a[NMAX + 2 * OFFSET_MAX];
int n, k, p, q, r;

inline int s(int i, int j) {
    if (i < 0 || j < 0) return 0;
    return a[i * k + j];
}

inline int getCoords(int x, int y) {
    return x * k + y;
}

int main() {
    ifstream fin("scuderia.in");
    ofstream fout("scuderia.out");

    fin >> n >> k >> p >> q >> r;

    if (p > k) p = k;

    int offset = k - r, totalSum = 0;

    for (int i = 0; i < n; ++i) {
        fin >> a[i + offset];
        totalSum += a[i + offset];
    }

    n += offset;
    int lastLine = n / k + (n % k != 0) - 1;

    for (int i = 0; i <= lastLine; ++i)
        for (int j = 0; j < k; ++j) {
            int ind = getCoords(i, j);
            if (i) a[ind] += a[getCoords(i - 1, j)];
            if (j) a[ind] += a[getCoords(i, j - 1)];
            if (i && j) a[ind] -= a[getCoords(i - 1, j - 1)];
        }

    int maxSum = INT_MIN, colGap = k - p;

    for (int i = q; getCoords(i, k - 1) < n; ++i) {
        // right aligned horizontal strip
        for (int j = p - 1; j < k; ++j) {

```

```

        int lineSum = s(i, k - 1) - s(i - q, k - 1),
            colSum = s(lastLine, j) - s(lastLine, j - p)
            ,
            intersectSum = s(i, j) - s(i - q, j) + s(i -
                q, j - p) - s(i, j - p);

        int cur = lineSum + colSum - intersectSum;
        if (cur > maxSum) maxSum = cur;
    }
    // l e f t a l i g n e d   h o r i z o n t a l   s t r i p   ( u s i n g   c o m p l e m e n t
    )
    for (int j = colGap; j < k - 1; ++j) {
        int hSum = totalSum -
            (s(lastLine, j) - s(lastLine, j -
                colGap)) +
            (s(i, j) - s(i - q, j) - s(i, j -
                colGap) + s(i - q, j - colGap));
        if (hSum > maxSum) maxSum = hSum;
    }
}

fout << maxSum << '\n';
return 0;
}

```